

SIMATS ENGINEERING

CO5 - ASSESSMENT TOOL 3

Technical Presentation

COURSE NAME: Deep Learning

COURSE CODE: MLA0408

COURSE OUTCOME COVERED

CO5: Students will be able to analyze and explain advanced generative deep learning models including Autoencoders, Deep Belief Networks, Boltzmann Machines, Deep Boltzmann Machines and Generative Adversarial Networks. They will be able to compare their architectures, explain their learning mechanisms, and apply suitable generative models to real-world problems. (BTL-3)

Assessment Tool Weightage: 50%

QUESTIONS:

1. Autoencoder Variants

Prepare and deliver a technical presentation on Under complete Autoencoders and Regularized Autoencoders. Explain their architecture, encoding and decoding process, reconstruction loss, and the role of regularization. Illustrate how these models can be used for dimensionality reduction and feature learning.

2. Stochastic Encoders, Decoders and Contractive Encoders

Prepare a technical presentation explaining Stochastic Encoders and Decoders and Contractive Encoders. Describe their working principles, objectives, robustness characteristics, and differences from standard autoencoders. Demonstrate a suitable real-world application for each approach.

3. Deep Belief Networks and Boltzmann Machines

Develop a technical presentation comparing Deep Belief Networks and Boltzmann Machines. Explain their architecture, energy-based learning, hidden and visible units, training process, and feature-learning capabilities. Illustrate their use in a suitable real-world application.

4. Deep Boltzmann Machines

Prepare a technical presentation on Deep Boltzmann Machines, explaining their multilayer architecture, learning mechanism, interaction among visible and hidden layers, and advantages over conventional Boltzmann Machines. Discuss a practical application and justify why a Deep Boltzmann Machine is appropriate for the selected problem.

5. Generative Adversarial Networks

Create a technical presentation illustrating the working of Generative Adversarial Networks using the Generator and Discriminator framework. Explain the adversarial training process,

loss functions, generation of synthetic data, major challenges, and practical applications. Compare GANs with Autoencoders and justify when GANs are a more suitable choice.

Code of Conduct:

I T.Chaitra(192425217) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:T.Chaitra

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 Exemplary	Level 3 Proficient	Level 2 Developing	Level 1 Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

1. Autoencoder Variants

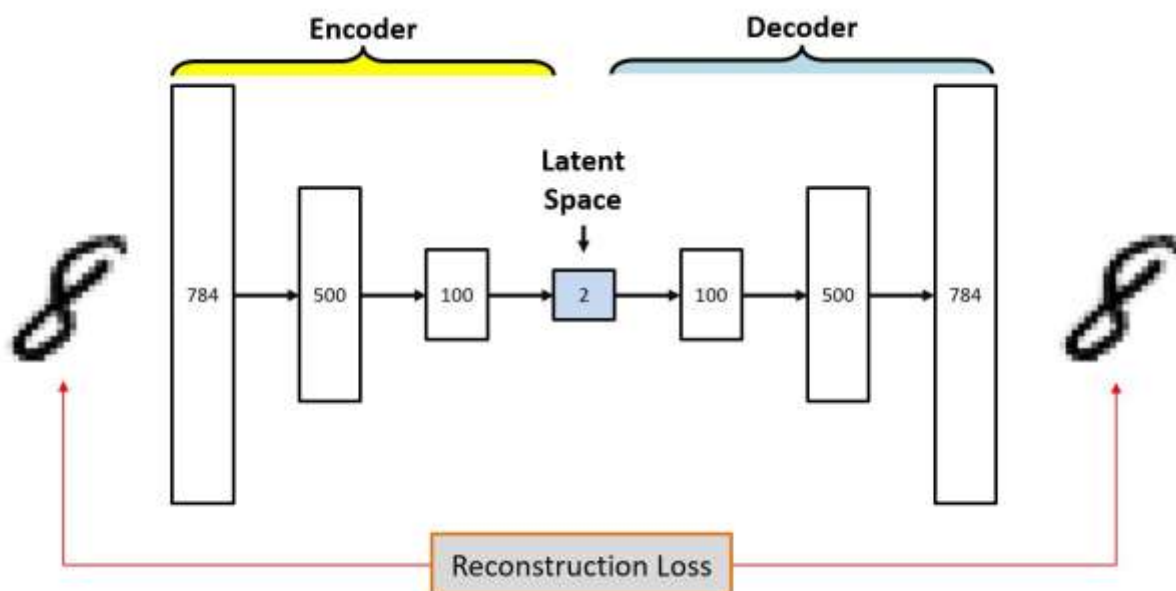
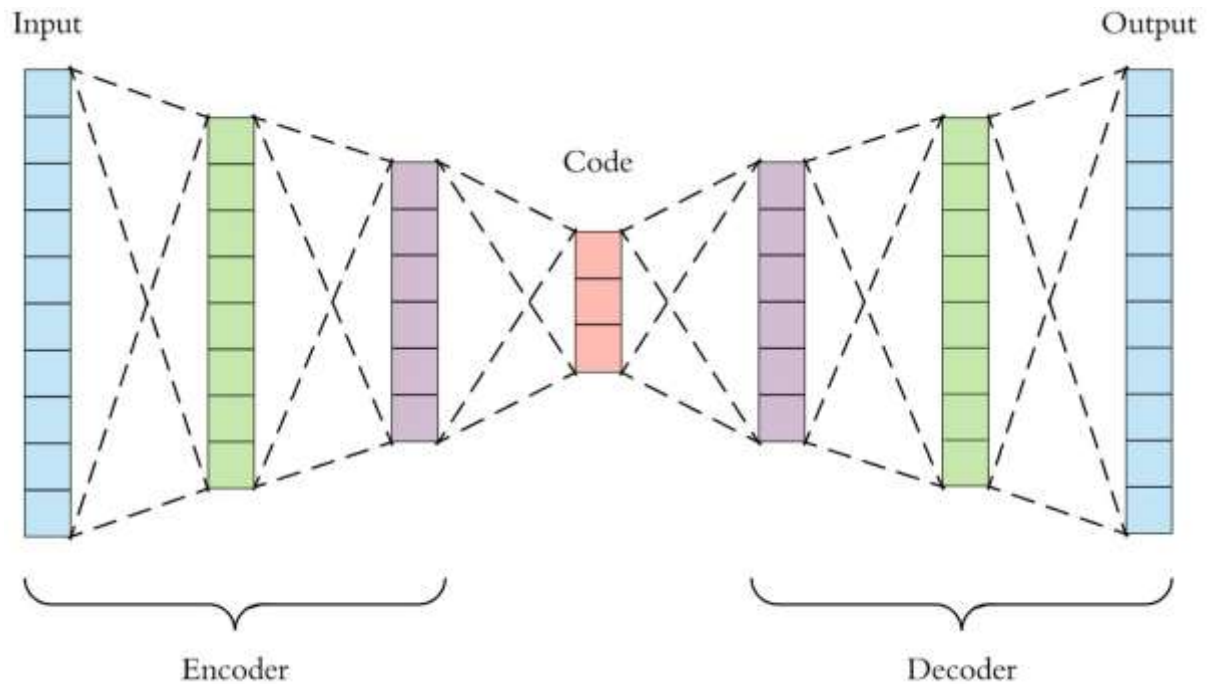
Undercomplete and Regularized Autoencoders

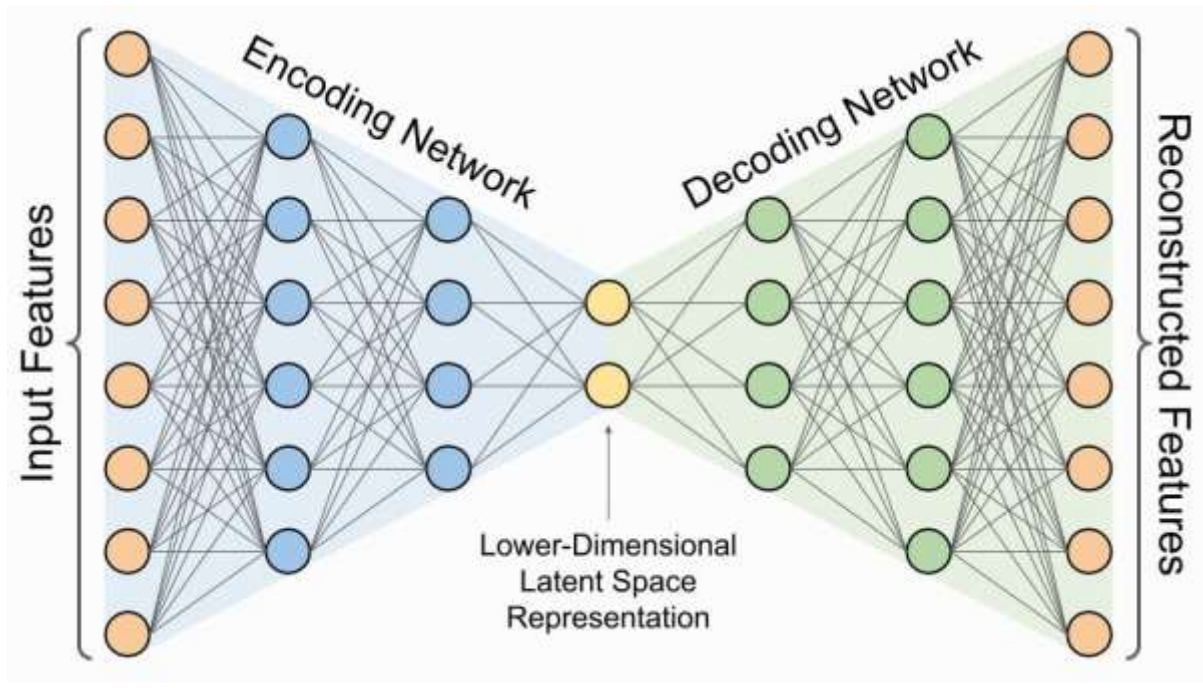
Introduction

An **Autoencoder (AE)** is an unsupervised neural network that learns a compact representation of input data and reconstructs the original input.

It has three major parts:

Input → Encoder → Latent Representation → Decoder → Reconstruction





5

Image 1 – Autoencoder architecture

Undercomplete Autoencoder

An **Undercomplete Autoencoder** has a latent representation with fewer dimensions than the input.

Example:

Input

100 features

↓

Encoder

64 → 32

↓

Latent Space

10 features

↓

Decoder

32 → 64

↓

Output

100 features

Encoding

The encoder converts input x into latent representation z :

$$z = f_{\theta}(x)$$

Decoding

The decoder reconstructs the input:

$$\hat{x} = g_{\phi}(z)$$

Reconstruction Loss

The model minimizes the difference between original and reconstructed data.

For MSE:

$$L = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2$$

Activation Functions

- Hidden layers: **ReLU**
- Output layer: **Linear** for continuous data
- **Sigmoid** can be used when input values are normalized to 0–1.

Dimensionality Reduction

If:

$$\text{Input} = 100 \text{ features}$$

and:

$$\text{Latent} = 10 \text{ features}$$

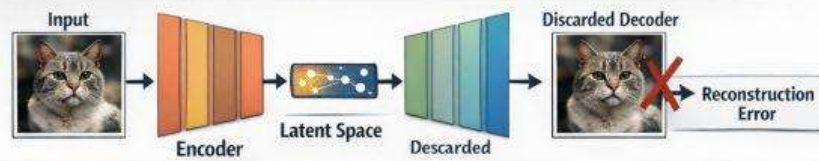
then the autoencoder has learned a **10-dimensional representation** of the original 100-dimensional data.

Regularized Autoencoder

A regularized autoencoder adds an additional constraint to prevent the model from simply memorizing the input.

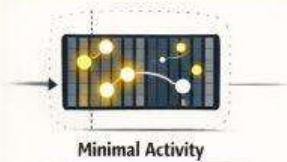
Regularized Autoencoders:

Learning Beyond the Identity



Sparse Autoencoder (SAE)

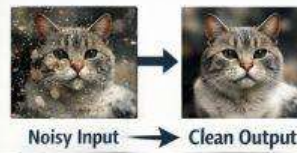
Sparse Representation



Encourages compact & interpretable features

Denoising Autoencoder (DAE)

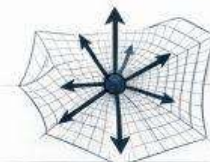
Robust Representation



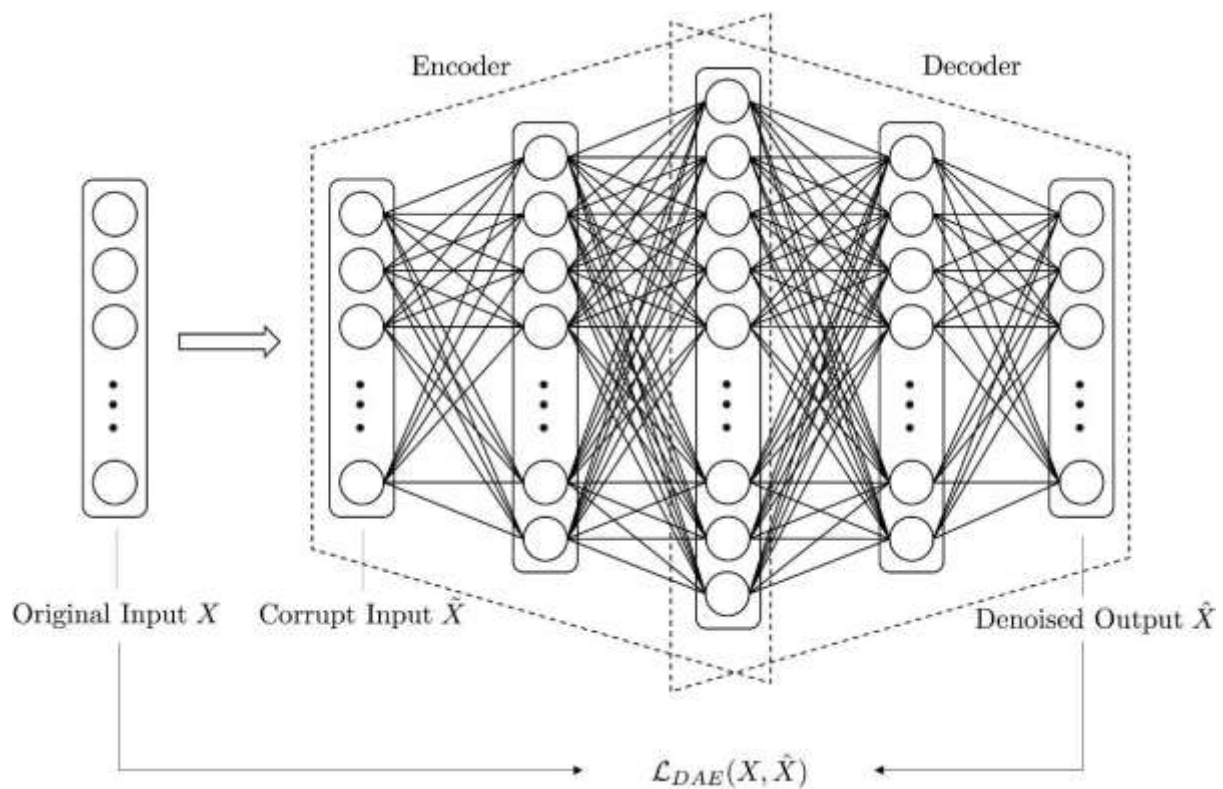
Recovers clean signal from corrupted input

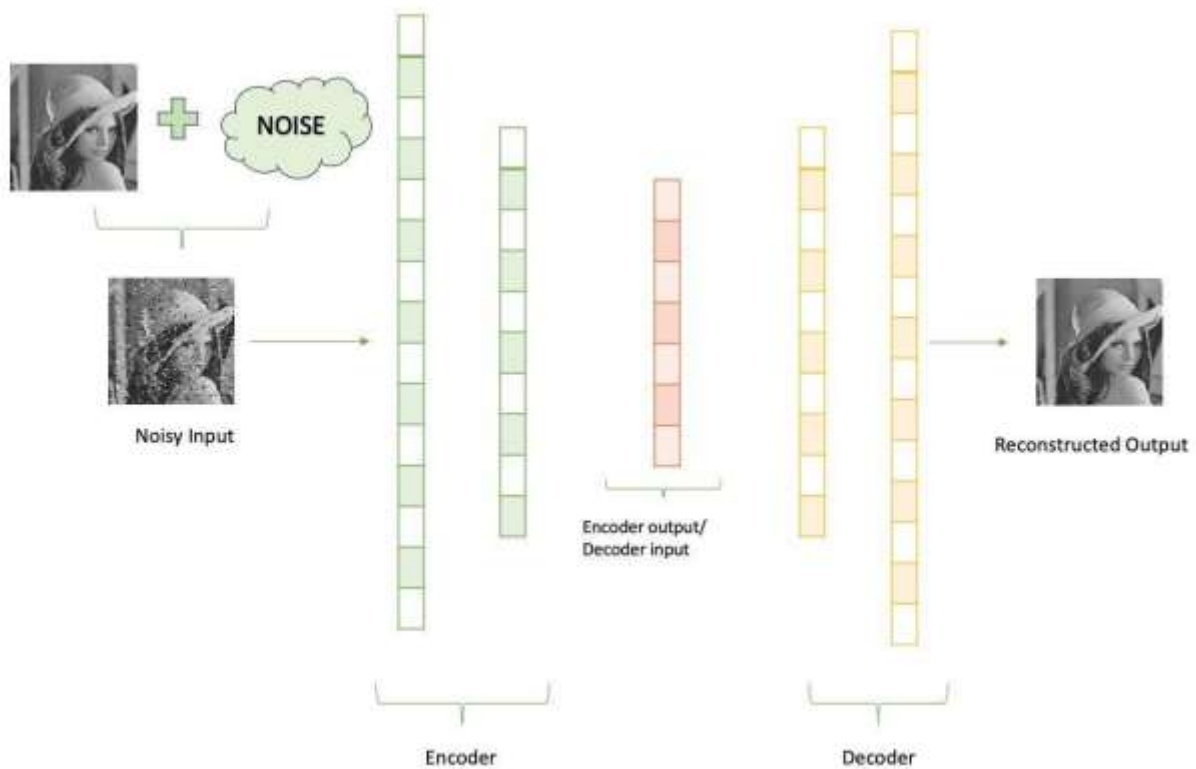
Contractive Autoencoder (CAE)

Stable Representation



Learns invariant & stable features





4

Image 2 – Regularized/Denoising Autoencoder

Common regularization approaches include:

1. Denoising Autoencoder

Add noise to input:

Clean Input → Add Noise → Encoder → Decoder → Clean Reconstruction

2. Sparse Autoencoder

Adds a sparsity penalty so only a small number of neurons become active.

3. Contractive Autoencoder

Adds a penalty that makes the representation less sensitive to small input changes.

Advantages

- Prevents overfitting.
- Learns useful features.
- Improves generalization.
- Handles noisy data.
- Useful for dimensionality reduction.

Applications

- Anomaly detection
- Image compression

- Feature extraction
- Noise removal
- Medical data analysis

Conclusion

Undercomplete autoencoders learn compact representations through a bottleneck, while regularized autoencoders impose additional constraints to learn more robust and meaningful features.

2. Stochastic Encoders, Decoders and Contractive Encoders

Introduction

Standard autoencoders learn a deterministic mapping:

$$x \rightarrow z \rightarrow \hat{x}$$

Stochastic and contractive models introduce additional mechanisms to improve representation quality and robustness.

Stochastic Encoder

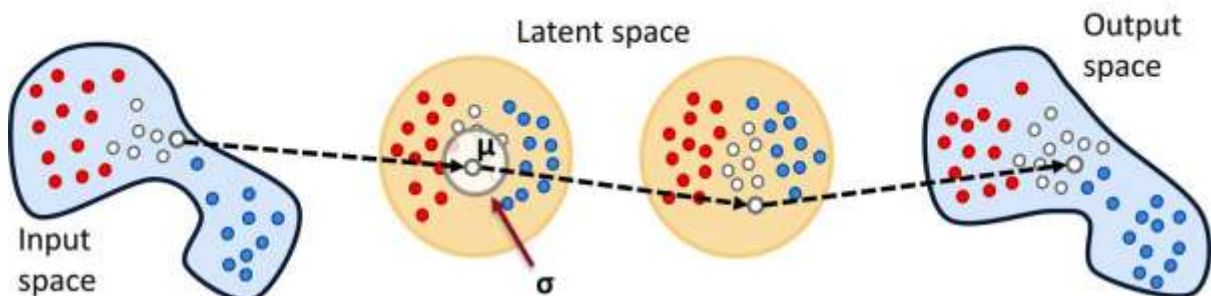
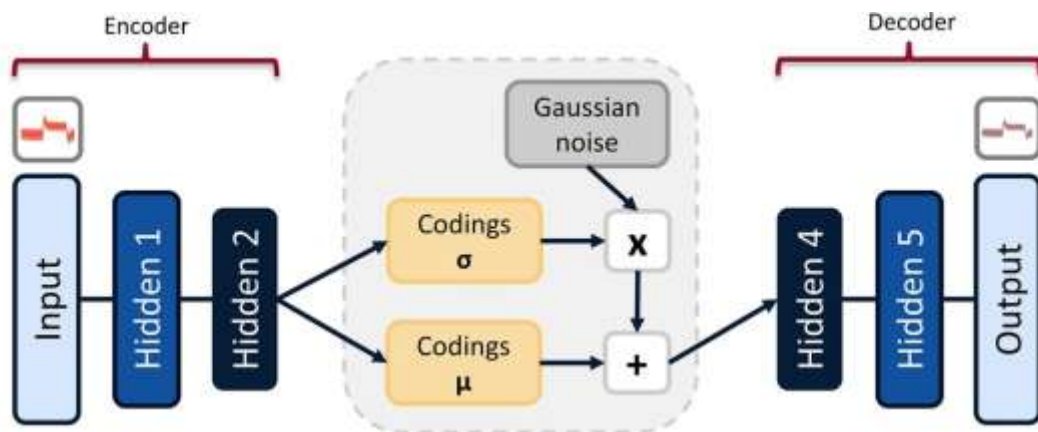
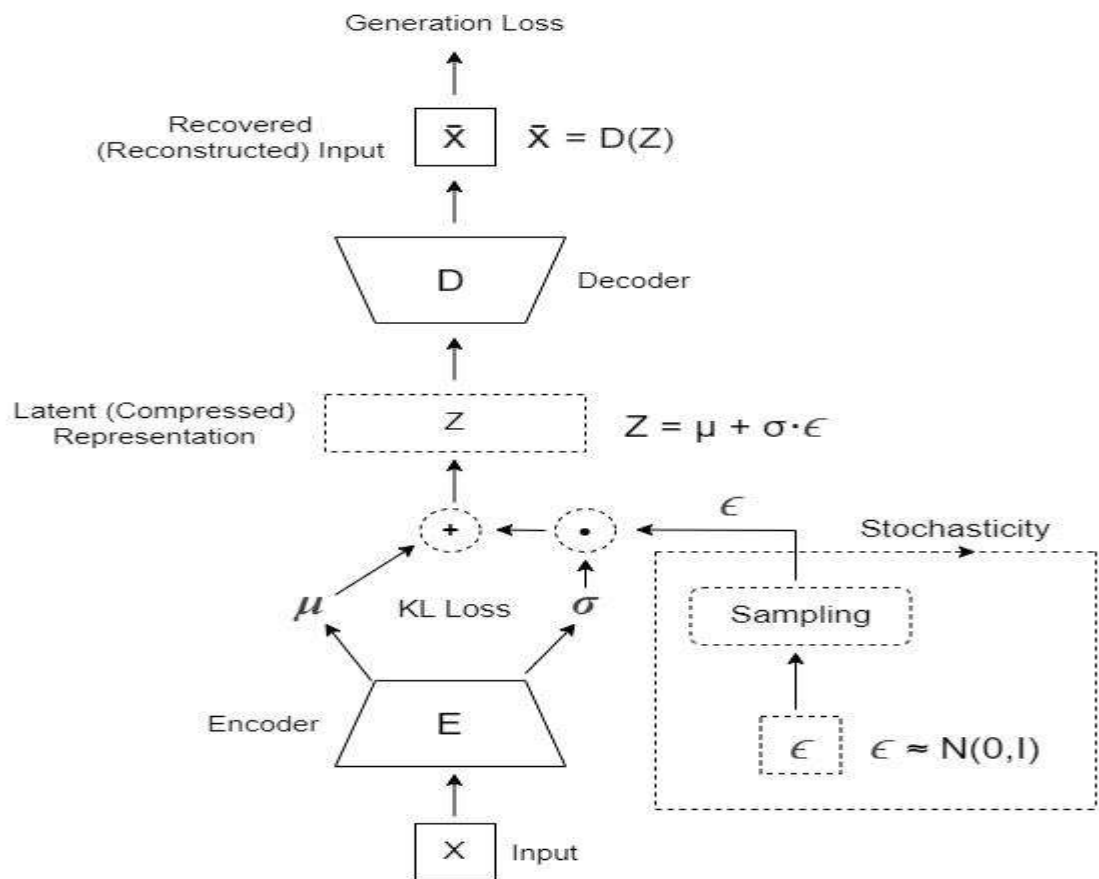
A stochastic encoder does not always produce exactly the same latent representation.

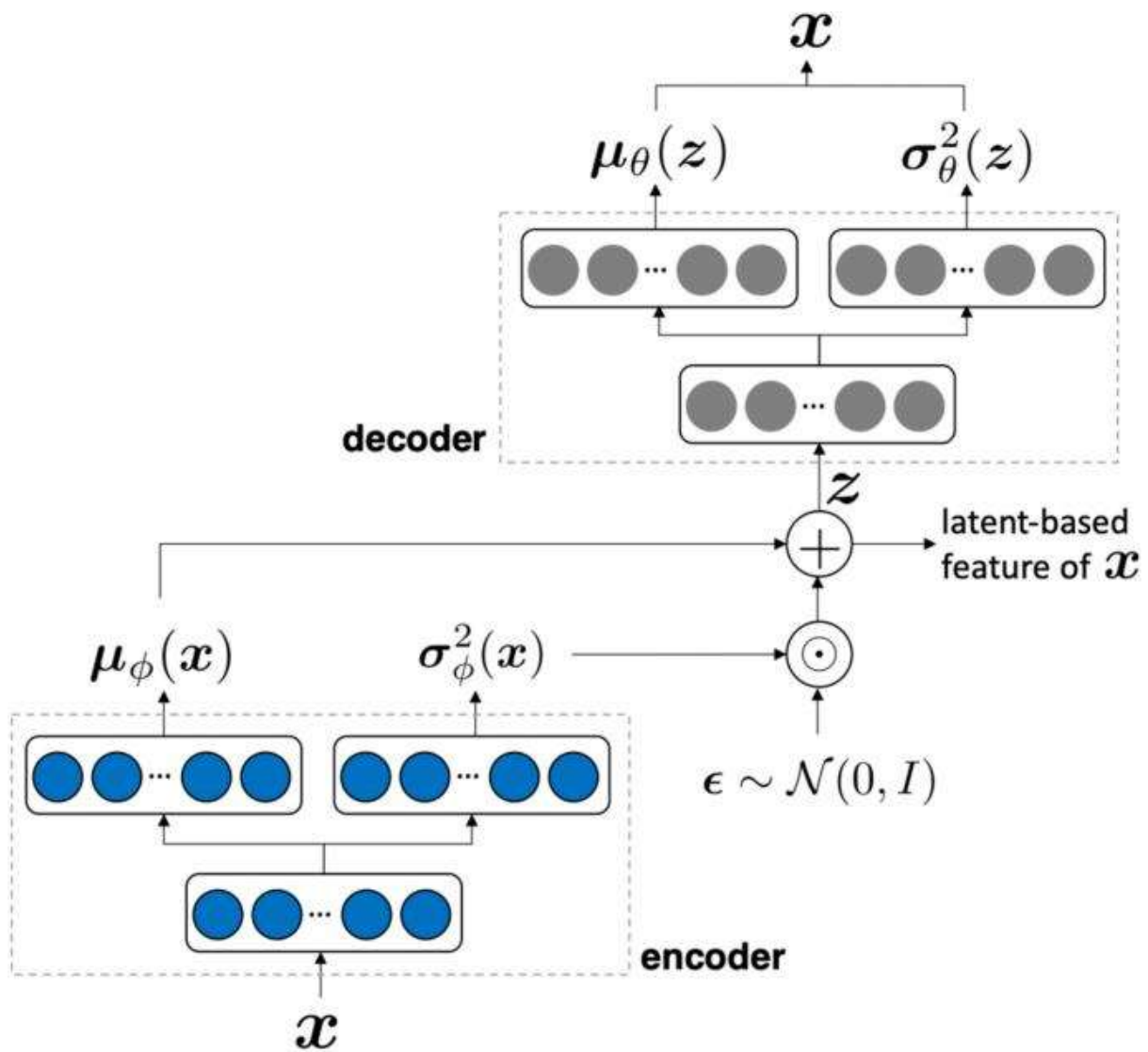
Instead, it learns a probability distribution:

$$q(z | x)$$

and samples:

$$z \sim q(z | x)$$





5

Image 1 – Stochastic Encoder/Decoder

Working

Input

↓

Encoder

↓

Mean μ + Variance σ

↓

Sampling

↓

Latent z

↓

Decoder

↓

Reconstruction

The stochastic representation makes the model capable of handling uncertainty.

Stochastic Decoder

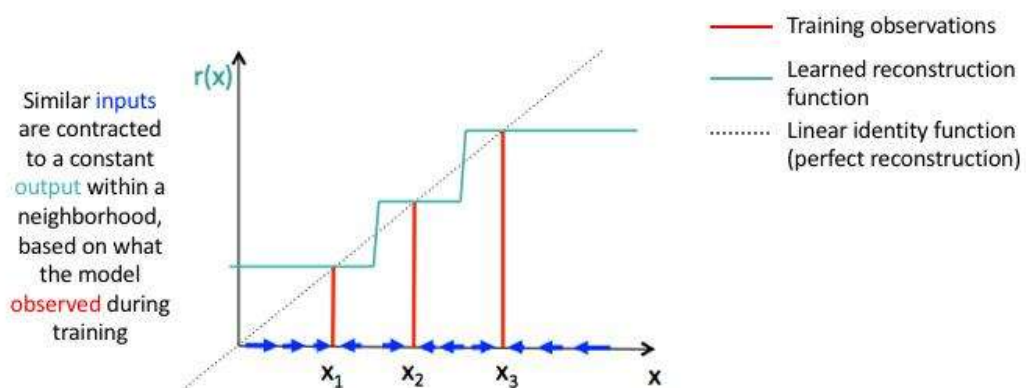
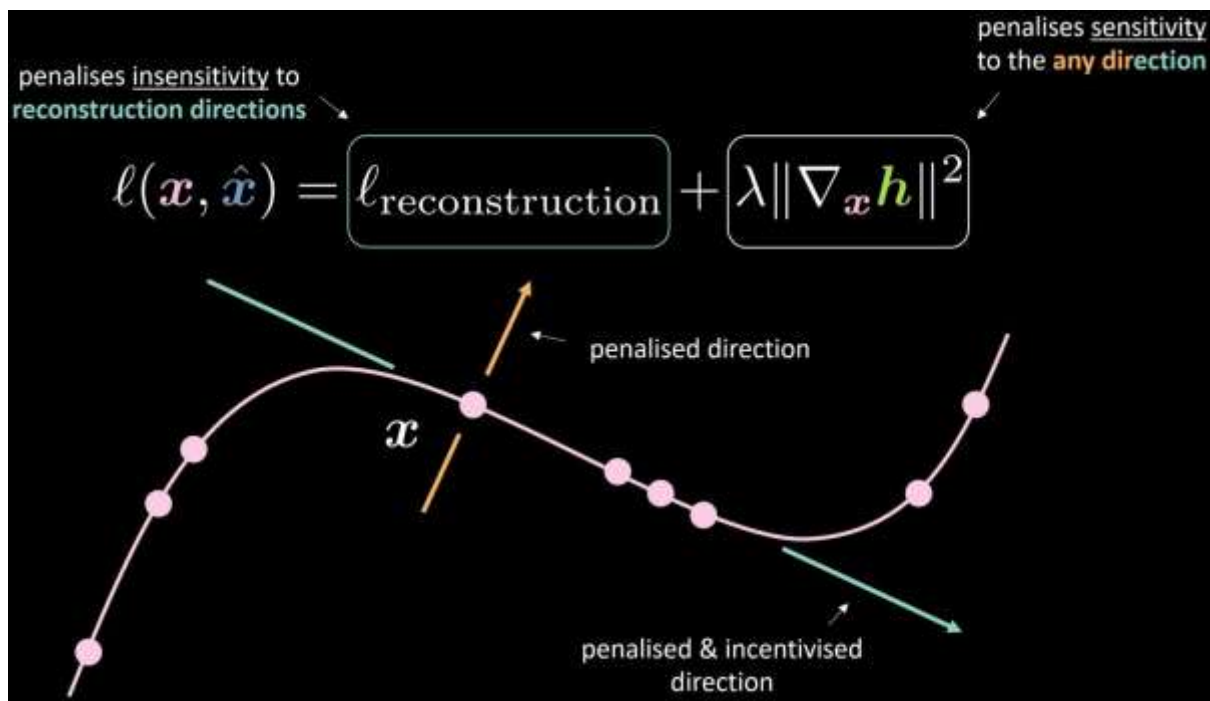
The decoder can also model a probability distribution over possible outputs:

$$p(x | z)$$

This is useful when multiple outputs may be possible for the same latent representation.

Contractive Encoder

A contractive encoder is designed to make representations stable against small input variations.



How Do Contractive Autoencoders Work?



5

Image 2 – Contractive Encoder

Objective

The loss consists of:

$$L = L_{reconstruction} + \lambda L_{contractive}$$

The contractive term penalizes excessive sensitivity of the encoder.

A common formulation is:

$$L_{contractive} = \|J_f(x)\|_F^2$$

where $J_f(x)$ is the Jacobian of the encoder.

Robustness

Original Image



Small Noise / Variation



Contractive Encoder



Similar Latent Representation

Comparison

Feature	Standard AE	Stochastic AE	Contractive AE
Representation	Deterministic	Probabilistic	Stable
Noise handling	Limited	Good	Good
Main objective	Reconstruction	Reconstruction + distribution	Reconstruction + stability
Key idea	Encode/decode	Sampling	Penalize sensitivity

Applications

Stochastic: image generation, uncertainty modeling.

Contractive: image recognition, signal processing, noisy sensor systems.

Conclusion

Stochastic encoders model uncertainty through probabilistic latent representations, whereas contractive encoders explicitly encourage stable representations under small input changes.

3. Deep Belief Networks and Boltzmann Machines

Introduction

Boltzmann Machines (BM) and **Deep Belief Networks (DBN)** are generative neural models that learn probability distributions from data.

Boltzmann Machine

A Boltzmann Machine contains:

- Visible units
- Hidden units
- Symmetric connections

Visible Units

● ● ● ● ●

↕ ↕ ↕ ↕ ↕

Hidden Units

● ● ● ●

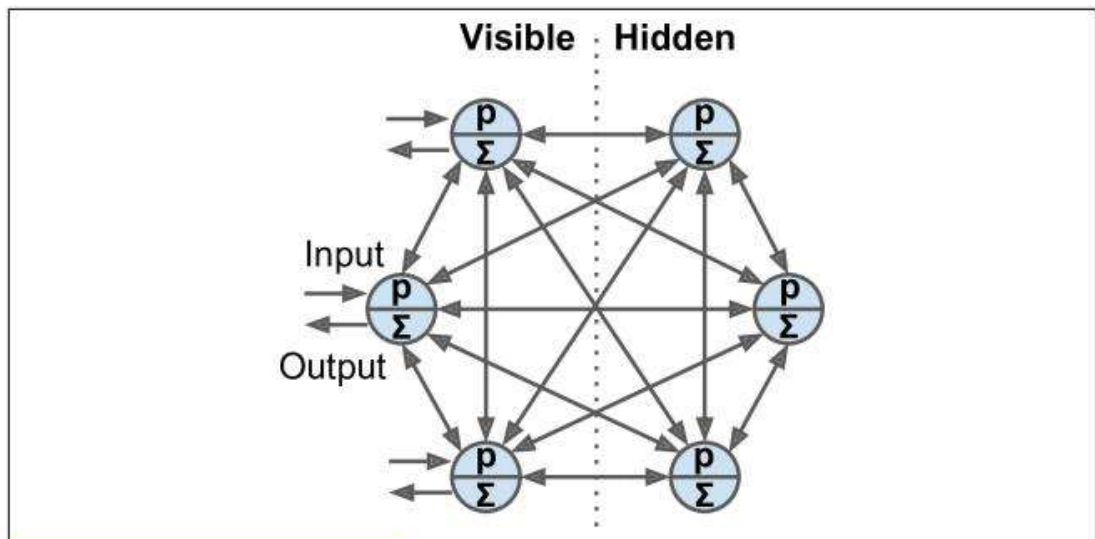
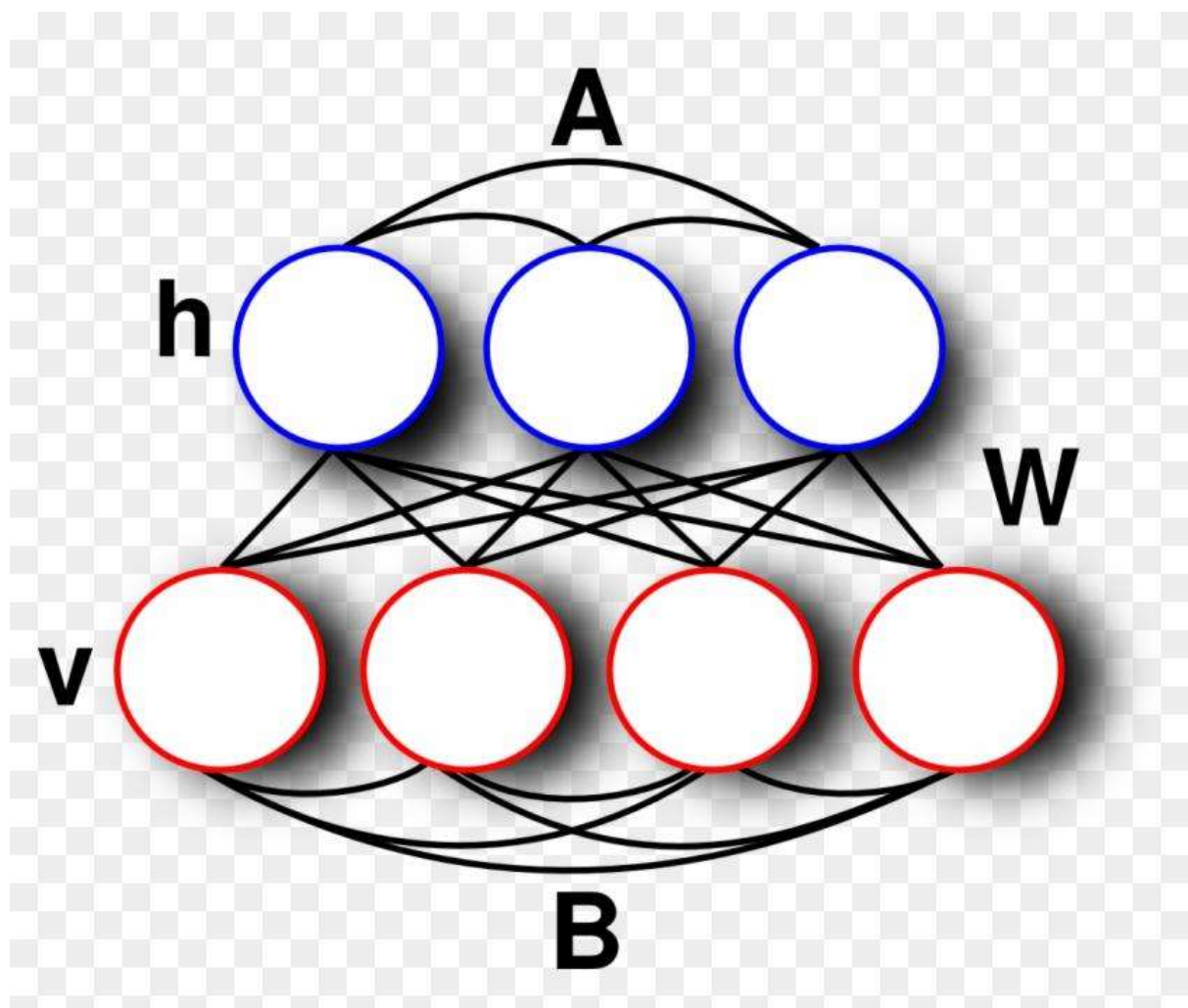


Figure E-2. Boltzmann machine



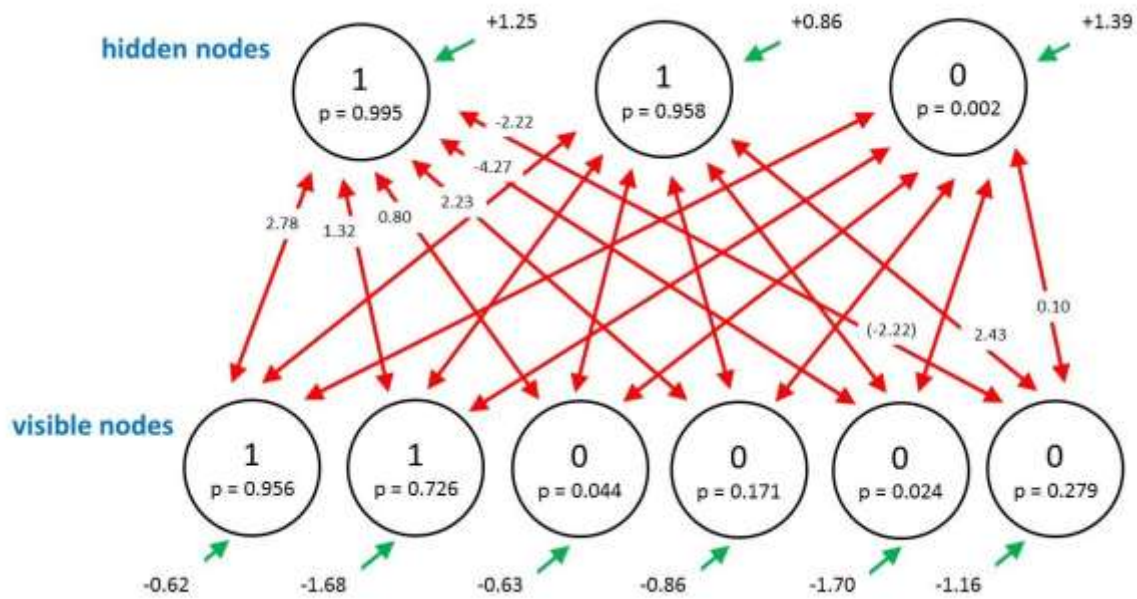


Image 1 – Boltzmann Machine architecture

Energy-Based Learning

A Boltzmann Machine assigns an energy to each configuration.

For a simplified BM:

$$E(v, h) = - \sum_i a_i v_i - \sum_j b_j h_j - \sum_{ij} v_i w_{ij} h_j$$

Lower-energy configurations are more probable.

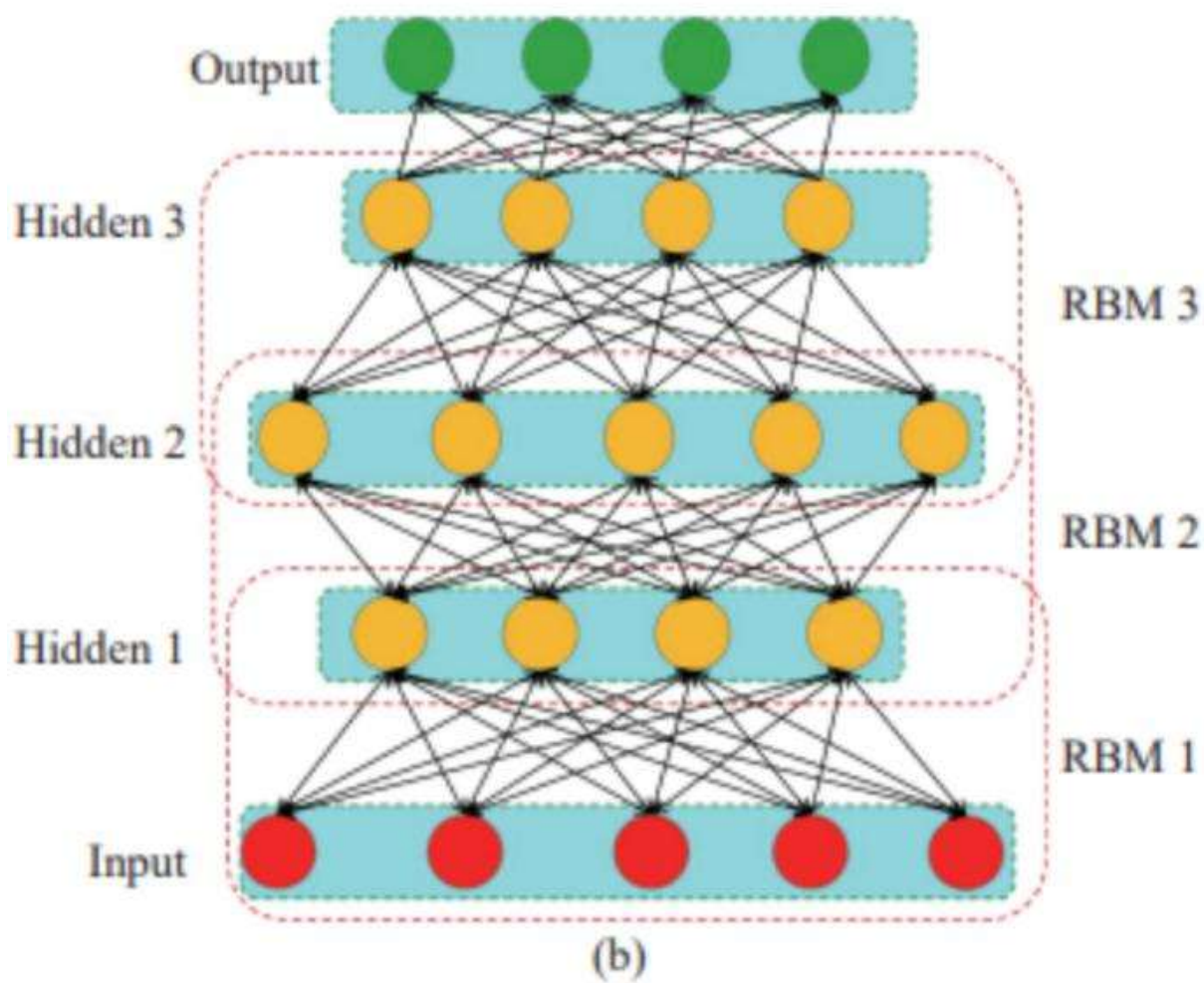
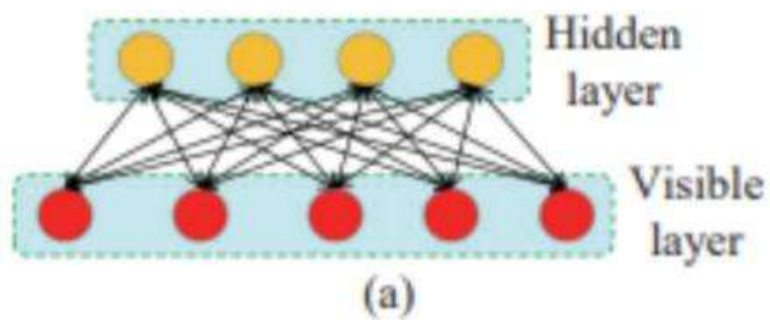
The probability is related to energy:

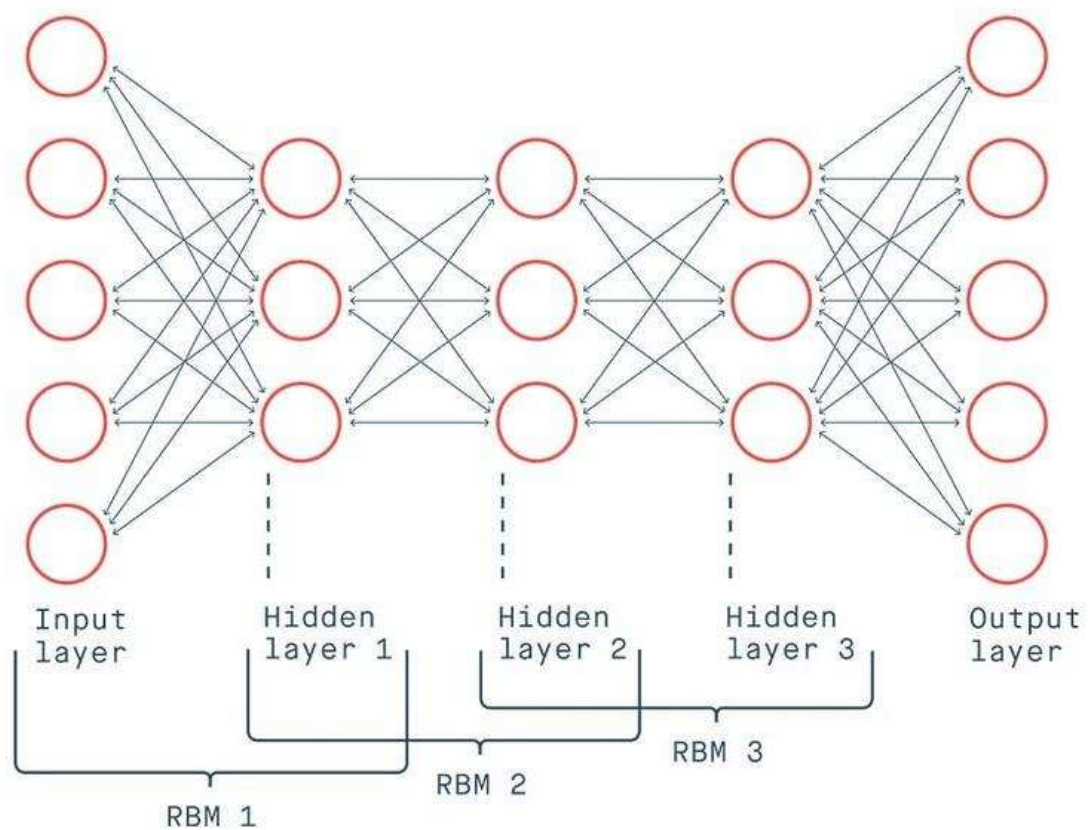
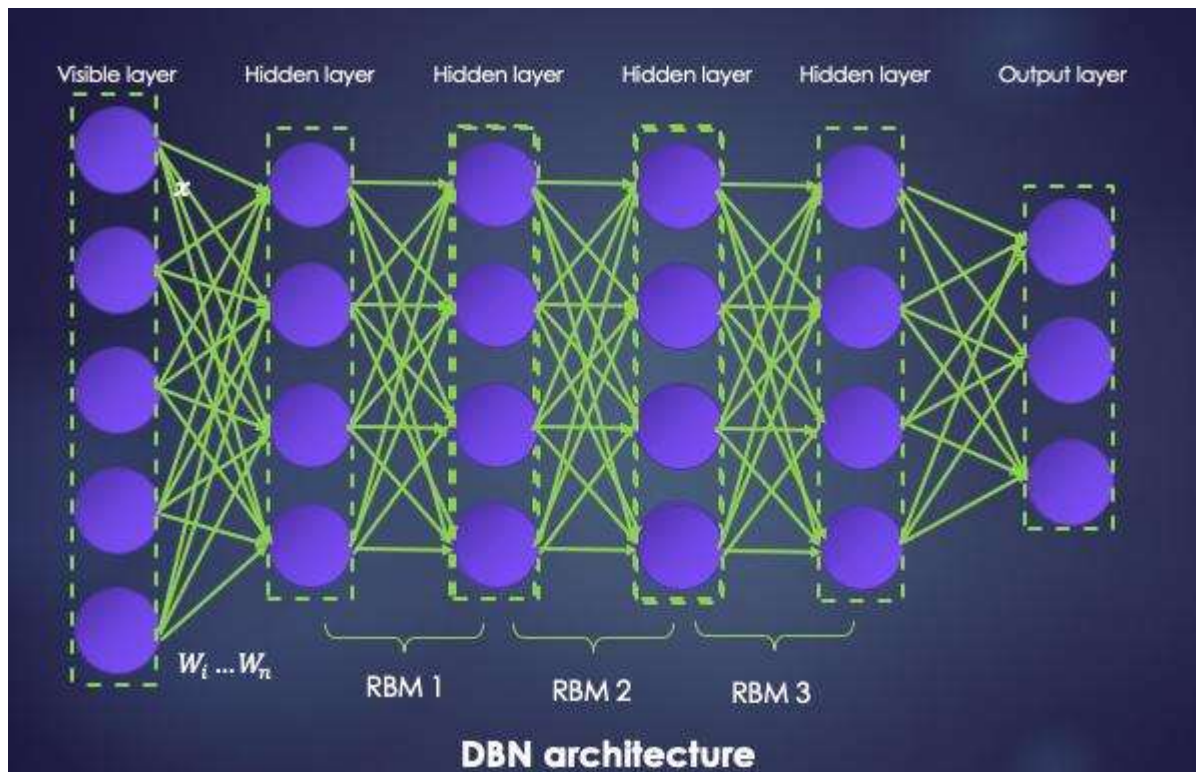
$$P(v, h) = \frac{e^{-E(v, h)}}{Z}$$

where Z is the partition function.

Deep Belief Network

A DBN is created by stacking multiple layers of hidden representations.





5

Image 2 – Deep Belief Network

Input / Visible Layer



Hidden Layer 1



Hidden Layer 2



Hidden Layer 3

Training Process

DBNs are commonly trained using **layer-wise unsupervised pre-training**.

Visible ↔ Hidden 1



Hidden 1 ↔ Hidden 2



Hidden 2 ↔ Hidden 3



Fine-tuning

Comparison

Feature	Boltzmann Machine DBN	
Architecture	Visible + hidden	Multiple hidden layers
Depth	Shallow	Deep
Feature learning	Basic	Hierarchical
Training	Energy-based	Layer-wise pretraining + fine-tuning
Representation	Less hierarchical	More hierarchical

Application

A DBN can be used for **recommendation systems**.

Example:

User Preferences



DBN



Hidden User Features



Recommendation

Advantages

- Unsupervised feature learning

- Learns hierarchical representations
- Useful when labeled data is limited
- Models complex probability distributions

Conclusion

A Boltzmann Machine provides energy-based probabilistic learning, while a DBN extends this idea into multiple layers and learns increasingly abstract features.

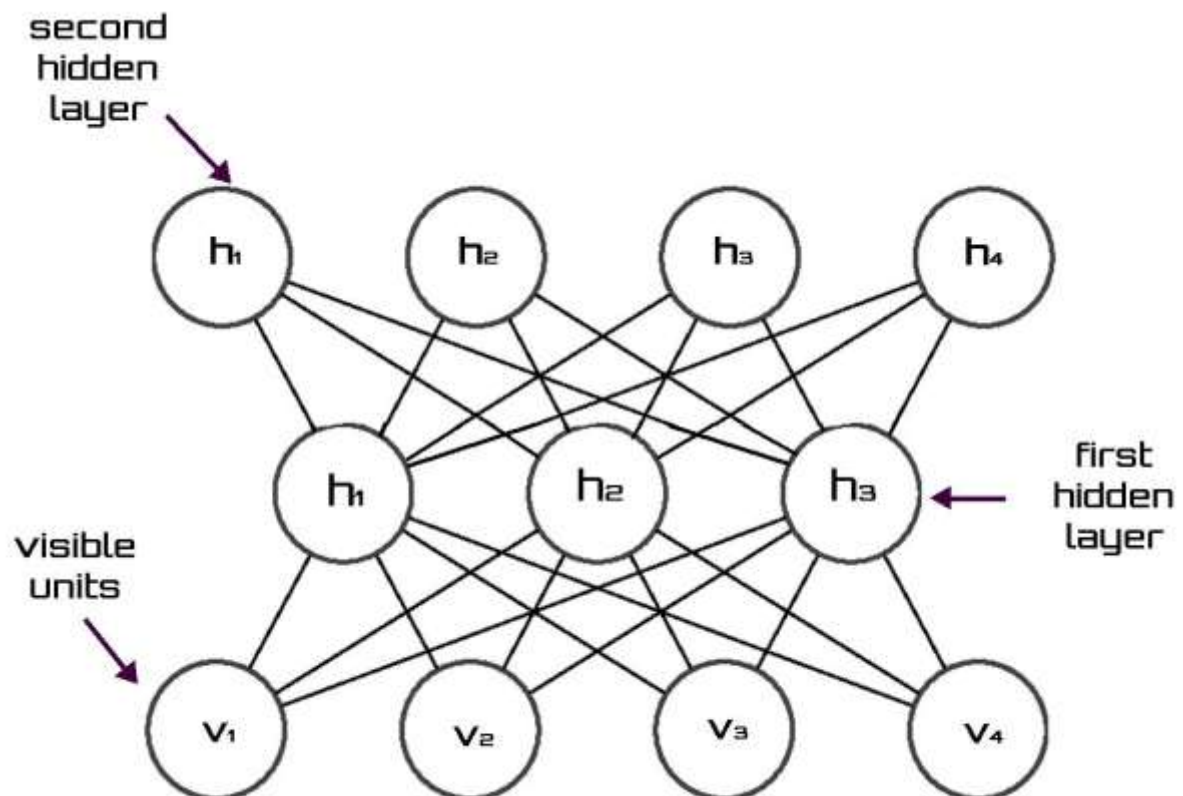
4. Deep Boltzmann Machines

Introduction

A **Deep Boltzmann Machine (DBM)** is an energy-based generative model containing multiple layers of hidden variables.

Unlike a conventional Boltzmann Machine, a DBM can learn **deep hierarchical representations**.

Architecture



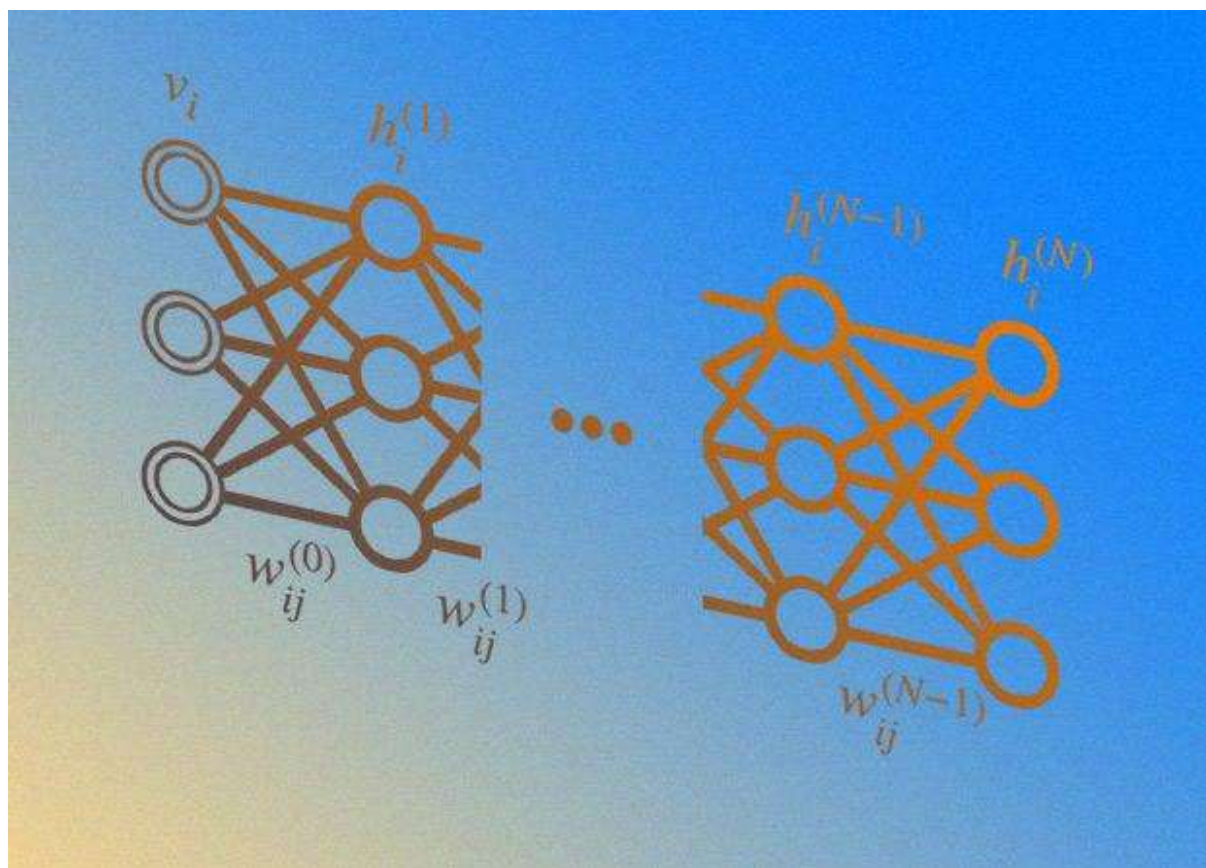
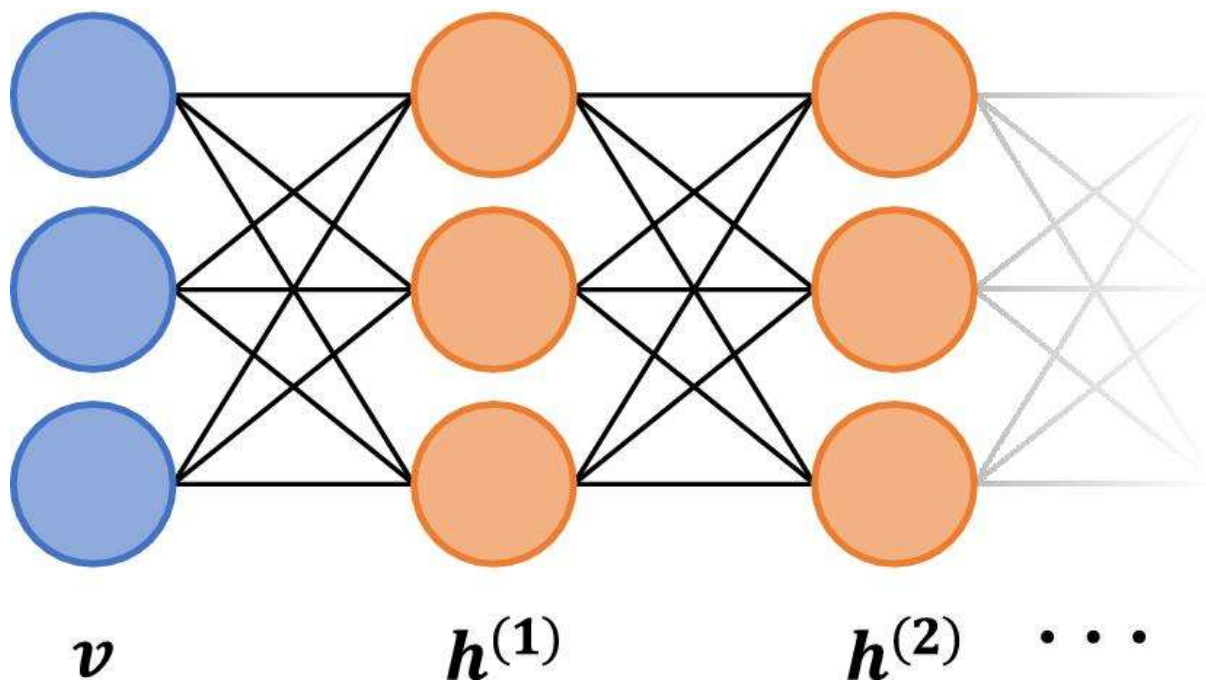
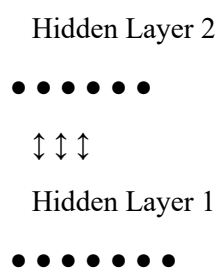
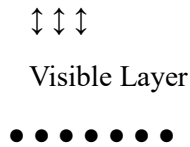


Image 1 – Deep Boltzmann Machine architecture





Layers

Visible layer: receives input data.

Hidden Layer 1: learns low-level features.

Hidden Layer 2: learns higher-level features.

More layers can learn increasingly complex representations.

Energy-Based Learning

The DBM assigns an energy to configurations of visible and hidden units.

A simplified energy function is:

$$E(v, h^1, h^2) = - \sum_i a_i v_i - \sum_j b_j h_j^1 - \sum_k c_k h_k^2 - \sum_{ij} v_i W_{ij} h_j^1 - \sum_{jk} h_j^1 W_{jk} h_k^2$$

The model learns parameters that make observed data more probable.

Learning Mechanism

Step 1

Initialize the network.

Step 2

Perform layer-wise pre-training.

Step 3

Use approximate inference to estimate hidden states.

Step 4

Update weights using approximate gradient learning.

Step 5

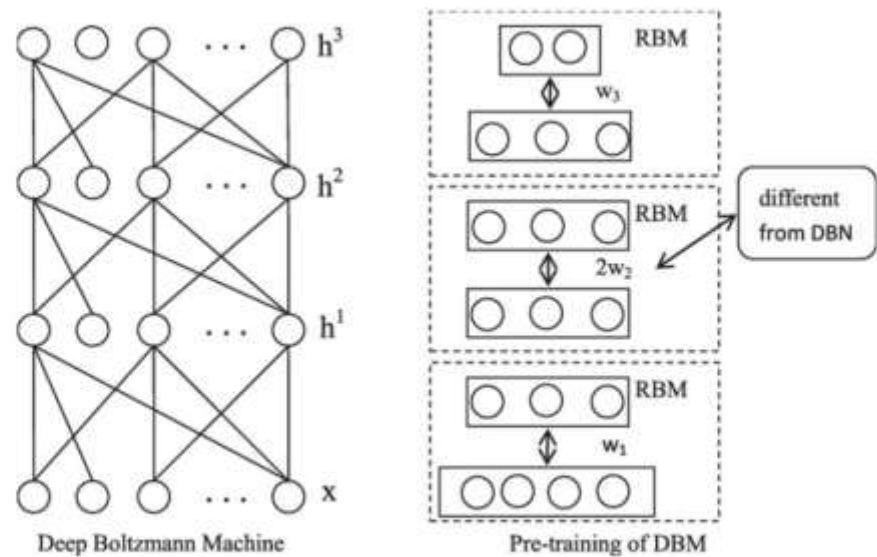
Fine-tune the complete model.

Advantages over Conventional BM

Conventional BM	Deep BM
Shallow	Multiple hidden layers
Simple features	Hierarchical features

Conventional BM	Deep BM
Limited representation	Rich representation
Less suitable for complex data	Suitable for complex data

Deep Restricted Boltzmann Machines



Deep Boltzmann Machine — Architecture & Mean-Field Inference

Structure

Visible ($L=3$)

Hidden 1 ($L=4$)

Hidden 2 ($L=2$)

W^1 (4×3)

W^2 (2×4)

$v = (0, 1)^3$

$h^{(1)} = \{0, 1\}^4 \Rightarrow \mu^{(1)} = \{0, 1\}^4$

$h^{(2)} = \{0, 1\}^2 \Rightarrow \mu^{(2)} = \{0, 1\}^2$

no intra-layer connections

Mean-Field Iteration

```

# Dimensions: v(4), μ1(3), μ2(2)
# W1: 4x3, W2: 3x2
v = [1, 0, 1, 0] # Apple, Premium
μ1 = [0.5, 0.5, 0.5] # init
μ2 = [0.5, 0.5] # init

for t in range(max_iter):
    # Update layer 1) messages from v AND μ2
    μ1 = σ(W1^T @ v + W2 @ μ2 + c1)
    # (3x4)@1 + (3x2)@1 + (3,1) = (3,1)
    # Update layer 2) messages from μ1 only
    μ2 = σ(W2^T @ μ1 + c2)
    # (2x3)@1 + (2,1) = (2,1)

    if max(Δμ) = ε: break
    # converged when activations stop changing

```

Result: belief = {μ1; μ2}

e.g. [0.92, 0.15, 0.78, 0.83, 0.61]

= passed to adapter = soft prompts

Key Property

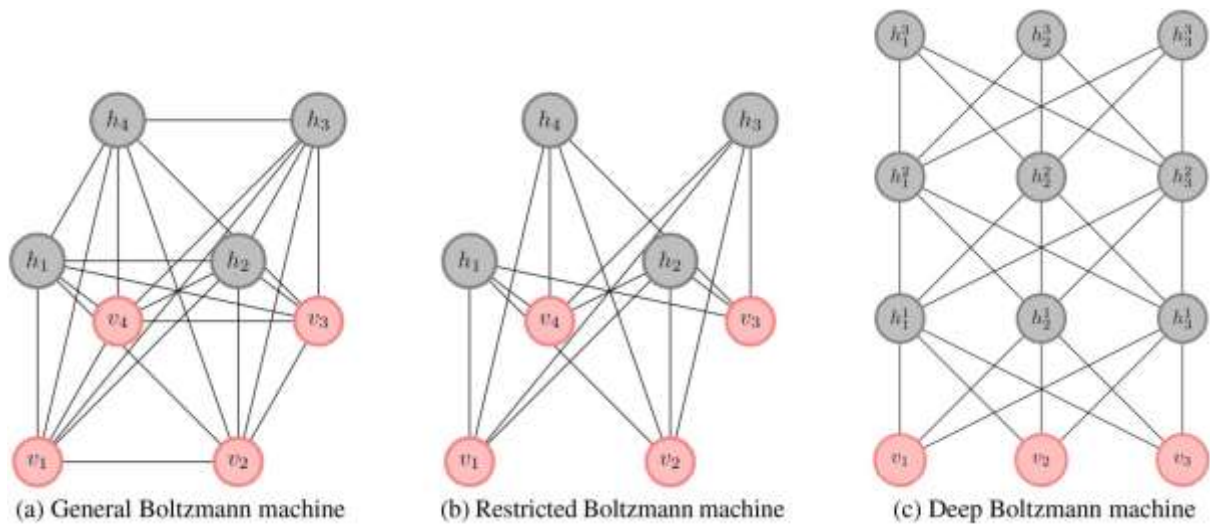
Each $\mu_i \in \{0, 1\}$ is the approximate probability that hidden unit i is ON.

$\sigma = \text{sigmoid}$ ensures this.

Convergence

Units keep adjusting based on neighbors until activations stabilize.

Typically 6-20 iterations.



4

Image 2 – DBM application/feature-learning example

Practical Application

Image recognition can use DBM for learning hierarchical image representations.

Image

↓

Visible Layer

↓

Hidden Layer 1

↓

Hidden Layer 2

↓

High-Level Representation

↓

Classification

Why DBM?

DBM is suitable when the application contains complex, high-dimensional data and requires hierarchical unsupervised feature learning.

Conclusion

A Deep Boltzmann Machine extends the conventional Boltzmann Machine using multiple hidden layers. It can learn complex hierarchical representations and is useful for feature learning, recommendation and pattern recognition.

5. Generative Adversarial Networks

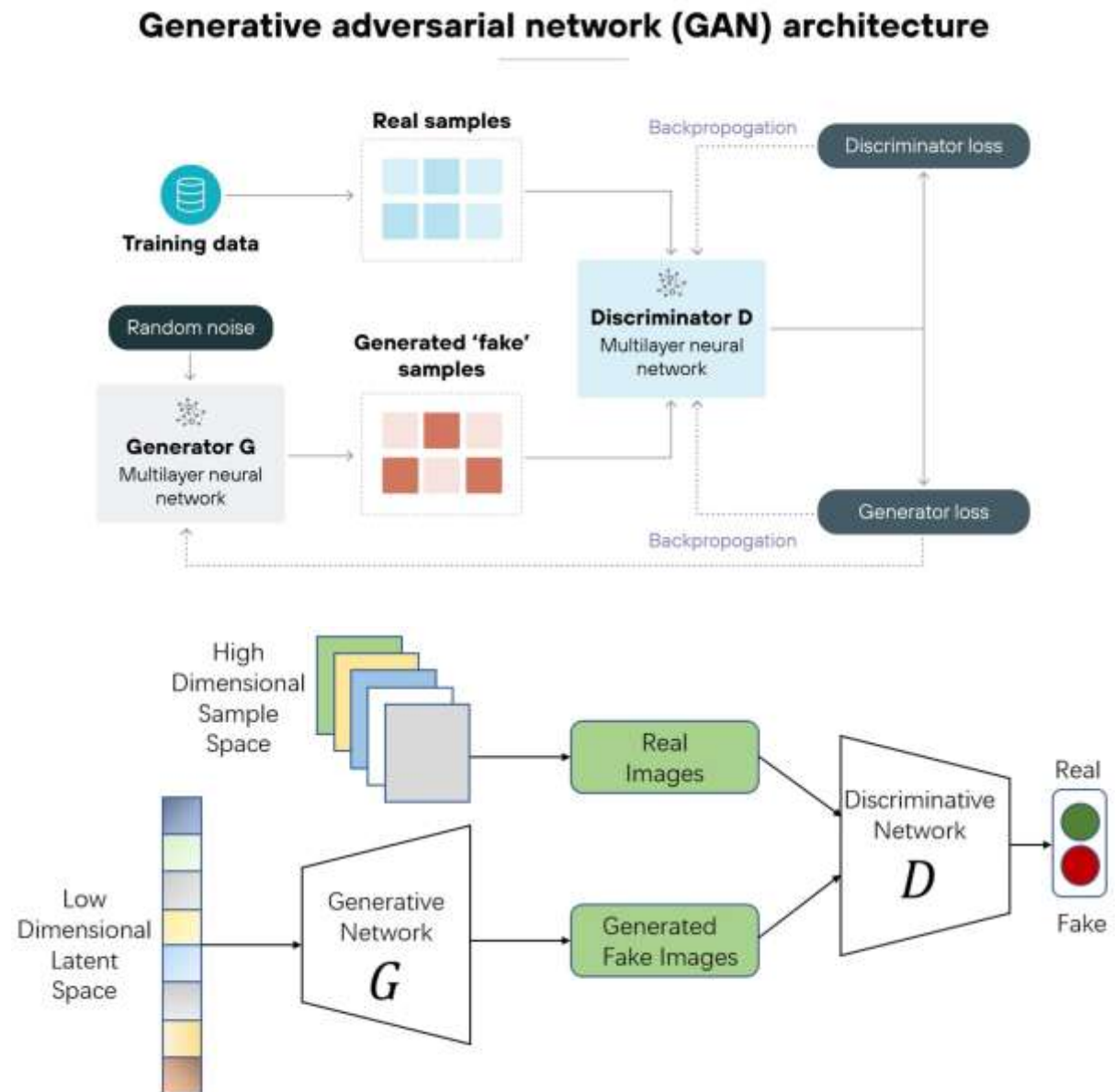
Introduction

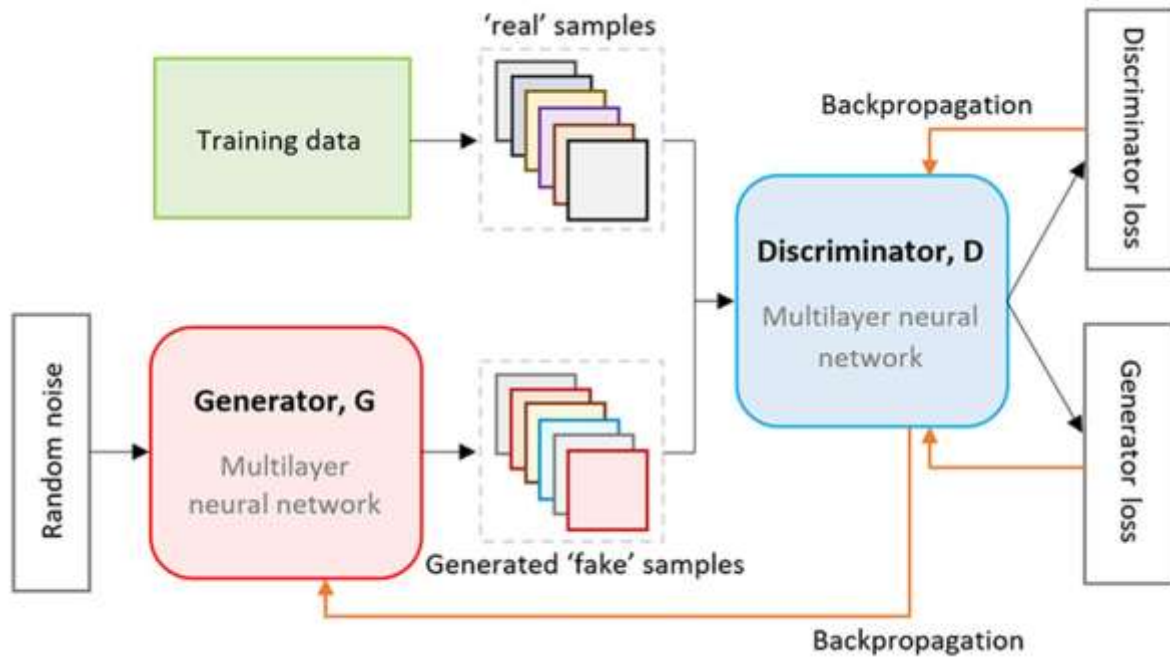
A **Generative Adversarial Network (GAN)** is a generative model containing two neural networks:

1. **Generator**
2. **Discriminator**

They compete with each other during training.

GAN Architecture





5

Image 1 – GAN Generator and Discriminator

Random Noise z



Generator



Fake Image



Discriminator ← Real Image



Real / Fake

Generator

The Generator converts random noise into synthetic data:

$$G(z)$$

Example:

Random Noise



Dense Layer



Upsampling



Convolution



Synthetic Image

Its goal is to produce realistic samples.

Discriminator

The discriminator receives real and generated samples.

Image



CNN



Feature Extraction



Classification



Real / Fake

Its goal is to distinguish real samples from fake samples.

Adversarial Training

The two networks compete.

Generator

Attempts to fool the discriminator.

Discriminator

Attempts to correctly identify fake images.

Noise



Generator



Fake Image



Discriminator



Feedback

↻

Generator improves

The original GAN objective is:

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p_z} [\log (1 - D(G(z)))]$$

Generation of Synthetic Data

After successful training:

Random Noise

↓

Generator

↓

Realistic Synthetic Data

The discriminator is generally not required when generating new samples.

Major GAN Challenges

1. Mode Collapse

The generator may produce similar samples repeatedly.

2. Training Instability

GAN training can be difficult to balance.

3. Vanishing Gradients

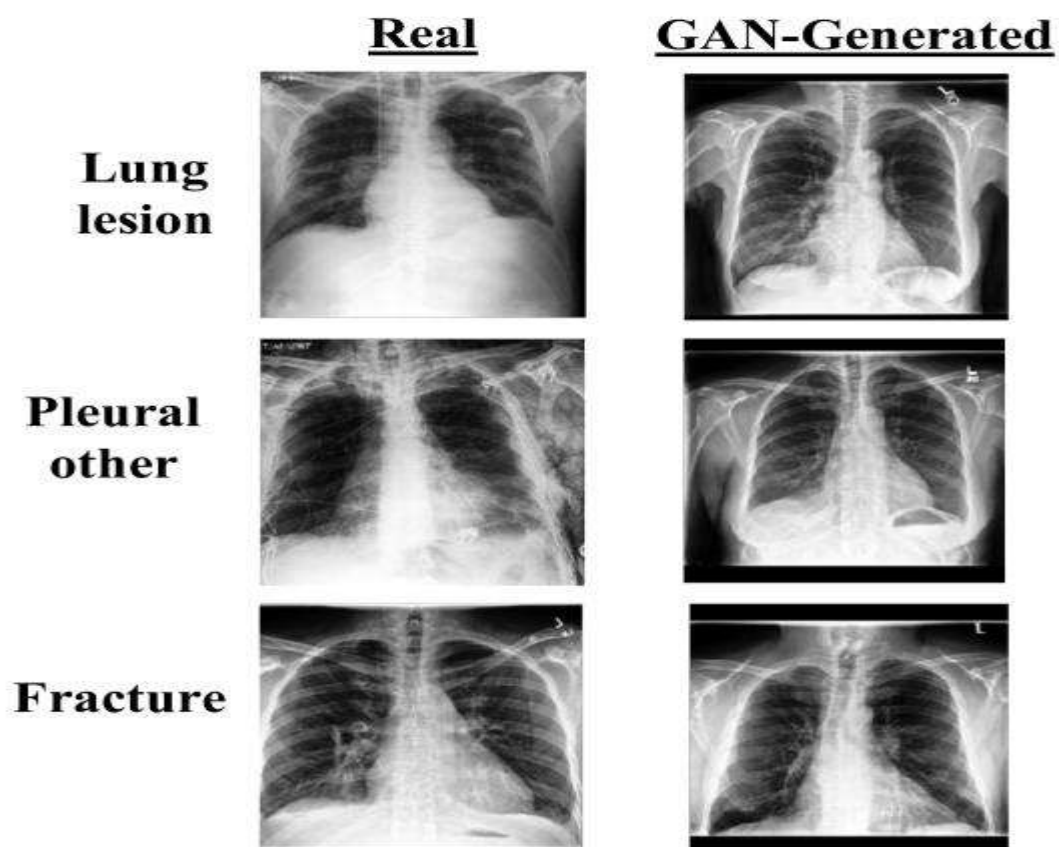
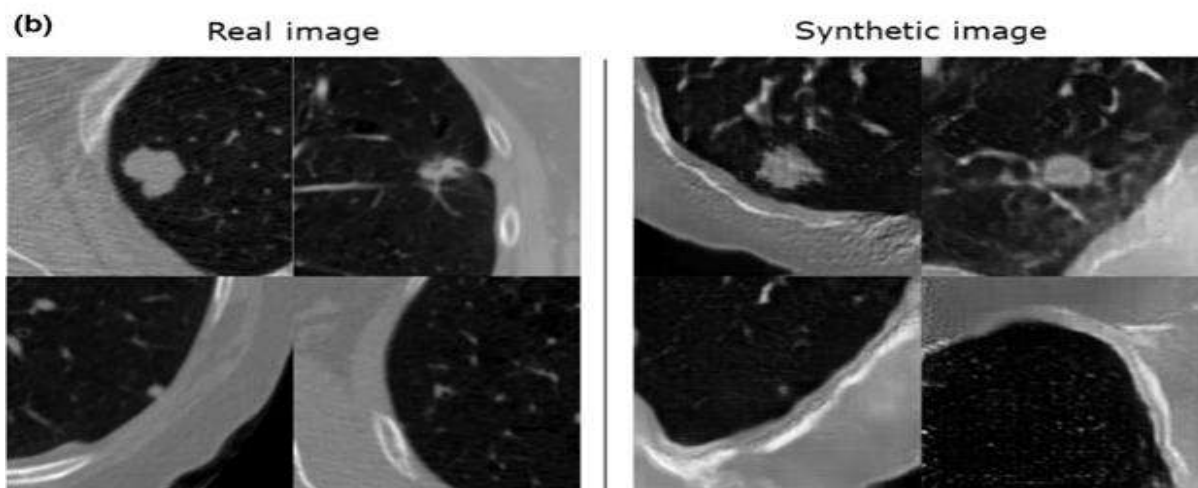
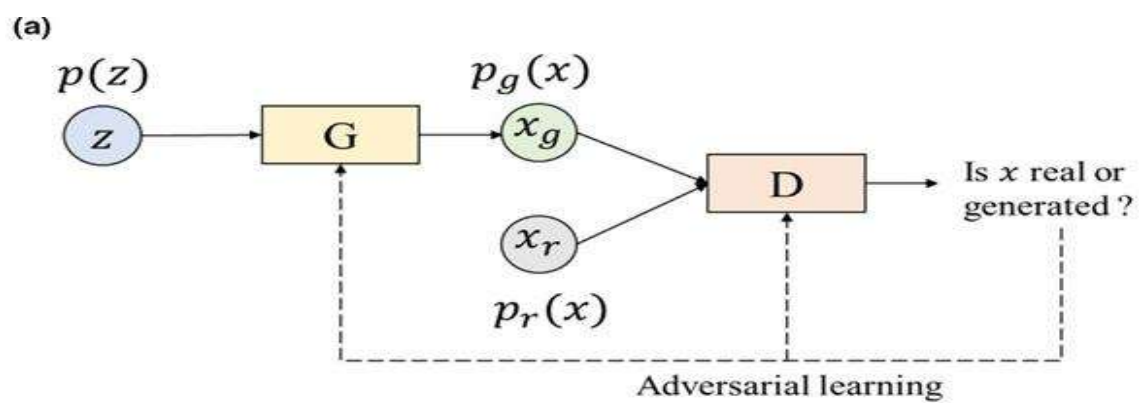
The generator may receive weak learning signals.

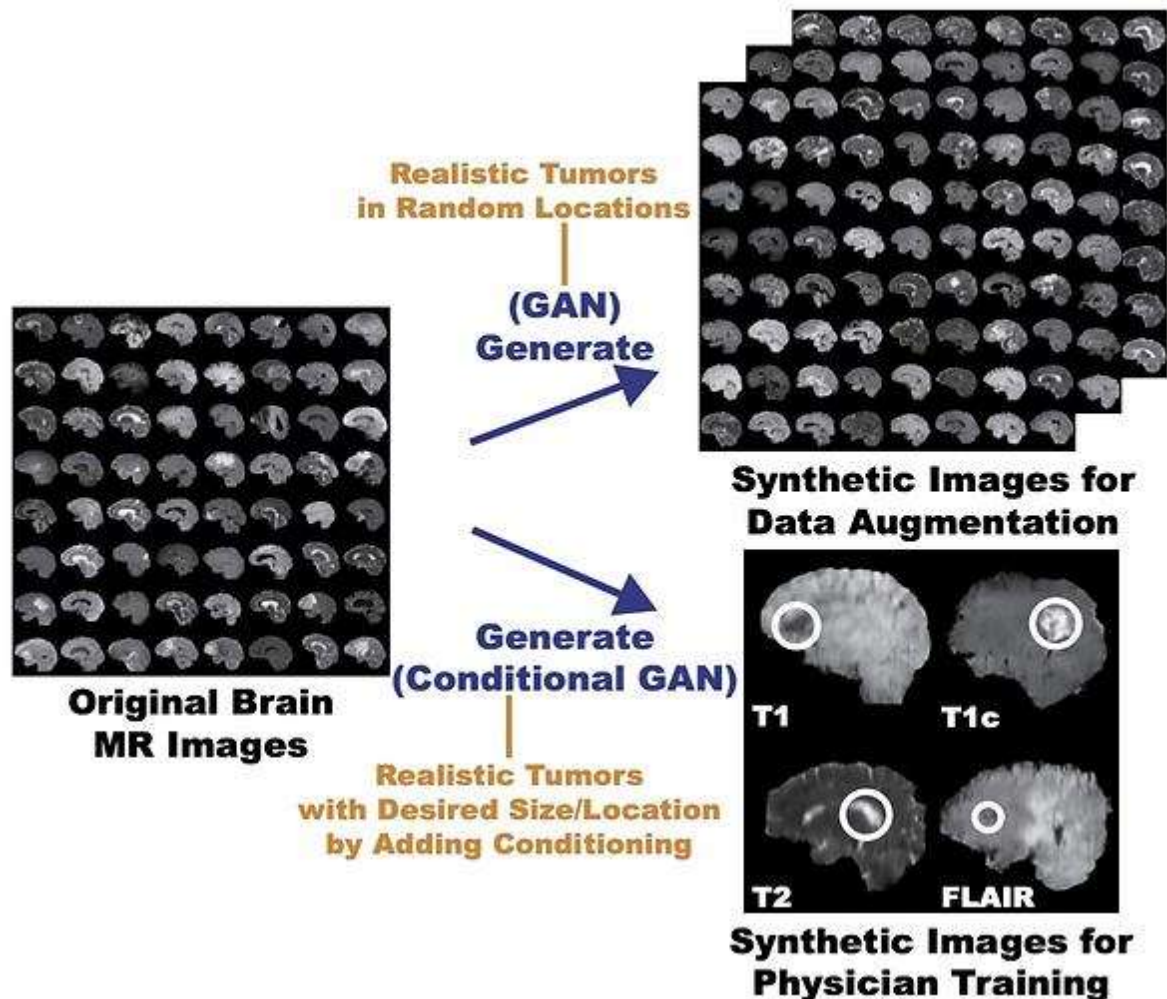
4. Evaluation Difficulty

Determining whether generated images are truly realistic can be challenging.

Applications

- Image generation
- Data augmentation
- Face synthesis
- Image-to-image translation
- Super-resolution
- Synthetic medical images
- Video generation





4

Image 2 – GAN-generated synthetic data/application

GAN vs Autoencoder

Feature	Autoencoder	GAN
Main purpose	Reconstruction	Data generation
Components	Encoder + Decoder	Generator + Discriminator
Loss	Reconstruction loss	Adversarial loss
Output	Reconstructed input	New synthetic samples
Image quality	May be blurry	Often sharper
Training	Relatively straightforward	More difficult
Best use	Feature learning/compression	Realistic data generation

Why choose GAN?

If an organization has **limited training images** and wants to generate **realistic new samples**, GAN is a strong choice.

For example:

Limited Images



GAN



Synthetic Images



Expanded Training Dataset



Better Downstream Model

Conclusion

GANs use adversarial competition between a Generator and Discriminator to learn realistic data distributions. They are especially useful for synthetic-data generation and data augmentation when realistic new samples are required.